

Reporte técnico

Pipeline de observabilidad end-to-end con OpenTelemetry

Microservicios en Python, OTel Collector, Jaeger, Prometheus, Loki y Grafana

Alcance	Laboratorio local reproducible con Docker Desktop
Flujo	k6 -> service-a -> service-b -> PostgreSQL
Señales	Trazas, métricas y logs estructurados correlacionados por trace_id
Evidencia	Ejecución local sin costos de GCP ni AWS

Resultado ejecutivo

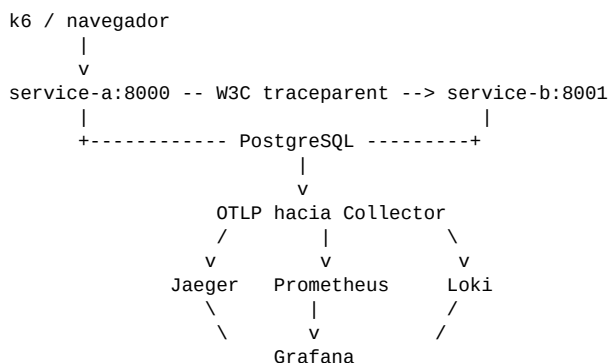
El laboratorio demuestra la propagación W3C del contexto desde service-a hacia service-b y PostgreSQL. El OTel Collector recibe las tres señales y las exporta a backends locales especializados. Las evidencias muestran 14 spans en una traza completa, métricas consultables en Prometheus, un dashboard operativo de seis paneles y logs JSON correlacionados mediante trace_id.

En el benchmark de 50 usuarios virtuales y cinco minutos de carga sostenida, ambas condiciones terminaron con 0 % de errores. Con OTel, el p99 aumentó 39.32 % y el throughput disminuyó 20.99 %, mientras que la memoria promedio aumentó solo 0.33 MiB en los dos servicios.

1. Objetivo y arquitectura

El objetivo es implementar un pipeline de observabilidad para dos microservicios con dependencia HTTP y acceso a base de datos. La solución prioriza la detección y resolución de problemas mediante correlación cross-signal.

Flujo de una solicitud



Componentes principales

Componente	Responsabilidad
service-a / service-b	API FastAPI, lógica de negocio, cliente HTTP, PostgreSQL y SDK OTel.
OTel Collector	Receiver OTLP gRPC/HTTP, processors memory_limiter/resource/batch y exporters de trazas, métricas y logs.
Jaeger	Almacenamiento y visualización local de trazas distribuidas.
Prometheus	Consulta y almacenamiento de métricas expuestas por el Collector.
Loki / Grafana	Almacenamiento, consulta y visualización de logs y métricas con enlaces hacia Jaeger.

Decisiones de diseño

El Collector desacopla los servicios de los backends, permitiendo cambiar Jaeger, Prometheus o Loki sin modificar la instrumentación. Jaeger usa almacenamiento en memoria para mantener el laboratorio local sin problemas de permisos ni costos. Las métricas de latencia usan buckets explícitos para que los percentiles sean interpretables. La infraestructura GKE y ECS Fargate está parametrizada como IaC, pero no se aplica durante esta entrega local.

2. Instrumentación y Collector

Los servicios utilizan el SDK de OpenTelemetry para emitir las tres señales. La instrumentación HTTP cubre las entradas FastAPI y las llamadas entre servicios; la instrumentación de PostgreSQL registra las consultas SQL. Además, se definieron spans de negocio para identificar validaciones, llamadas al servicio dependiente y accesos a datos.

Spans personalizados relevantes

```
order.business.validate
order.db.fetch
order.service_b.call
inventory.business.validate
inventory.db.fetch
SELECT
```

Los logs se emiten como JSON y contienen el contexto de la operación. En Loki, la serialización OTLP muestra los campos como traceid y spanid, conservando los identificadores W3C de 32 y 16 caracteres respectivamente. Grafana tiene un derived field que reconoce ambas variantes y permite abrir la traza en Jaeger.

Configuración del Collector

```
receivers: otlp (grpc :4317, http :4318)
processors: memory_limiter -> resource -> batch
exporters: jaeger, prometheus, loki, debug
metrics endpoint: :8889
health endpoint: :13133
```

La evidencia collector-healthy confirma que el Collector procesa periódicamente spans y métricas. En la ejecución observada se registraron resource spans, 14 métricas y 22 puntos de datos por ciclo de exportación, sin errores recientes.

Evidencia de Jaeger

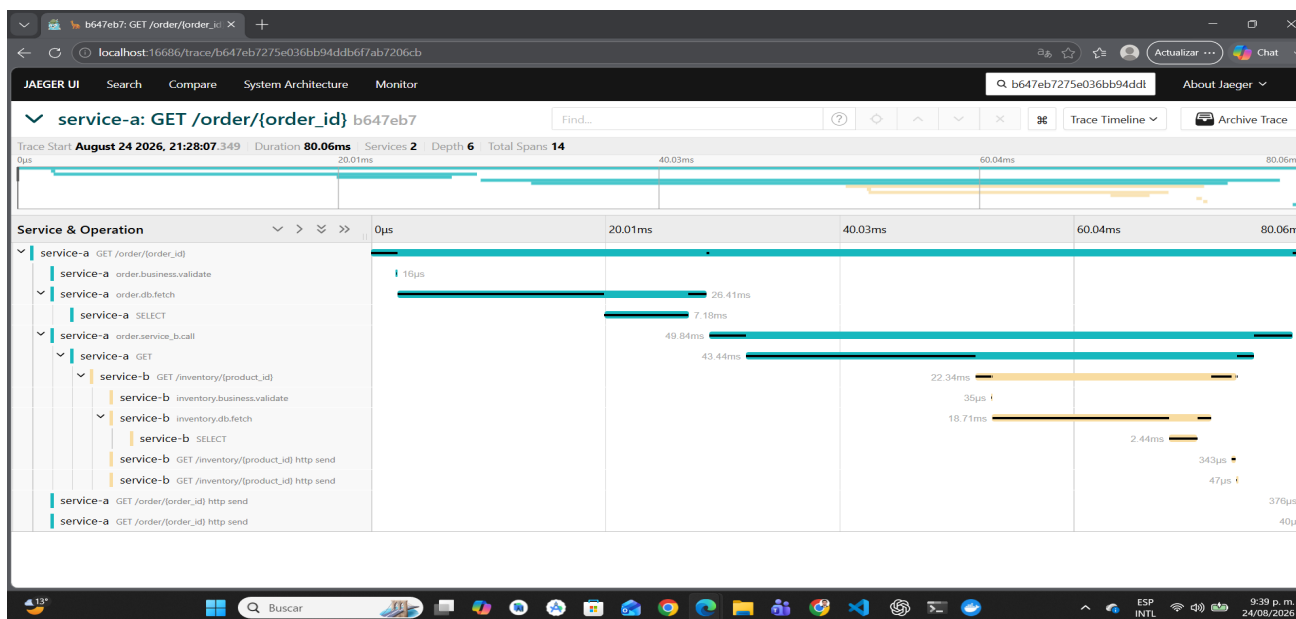


Figura 1. Traza distribuida con service-a, service-b, PostgreSQL y 14 spans.

3. Métricas y visualización

Prometheus consulta el endpoint de métricas del Collector y permite comprobar las series emitidas por cada servicio. La evidencia directa muestra 15 solicitudes para cada microservicio y valores de p95 numéricos para ambos.

Resultados Prometheus

Métrica	service-a	service-b
Solicitudes	15	15
p95	75 ms	24.25 ms

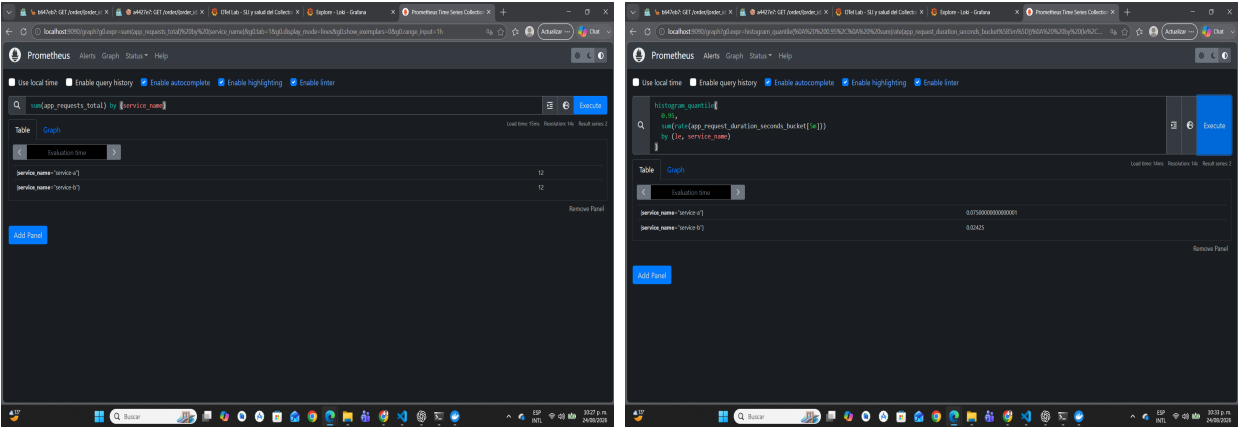


Figura 2. Consultas directas en Prometheus: solicitudes por servicio y latencia p95.

Dashboard Grafana

El dashboard reúne cuatro SLIs - latencia p99, tasa de error, throughput y disponibilidad - y dos indicadores operativos del Collector - CPU y errores o datos descartados. La ejecución observada presentó 100 % de disponibilidad, 0 % de errores y cero spans descartados.

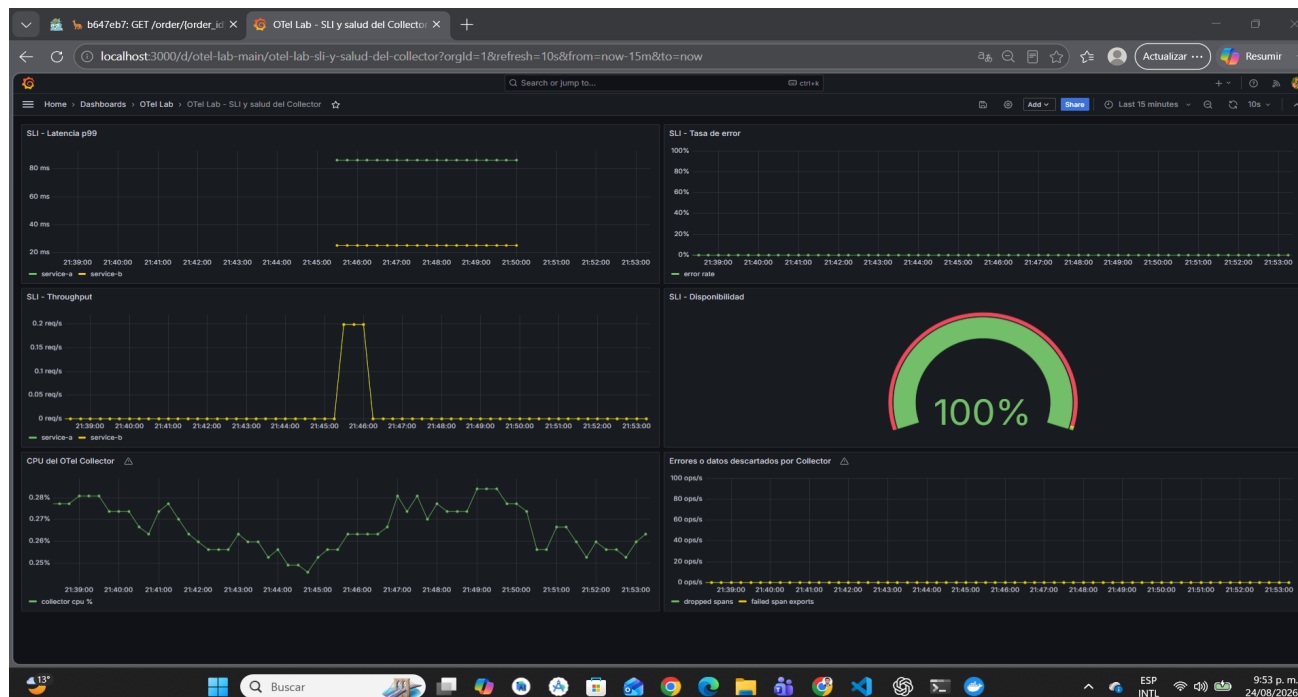


Figura 3. Dashboard Grafana con seis paneles y datos locales.

4. Logs estructurados y correlación cross-signal

Loki recibe los logs JSON a través del Collector. La consulta por etiquetas permite localizar ambos servicios y la consulta de contenido permite filtrar una única traza. El mismo traceid aparece en las tres operaciones: procesamiento del pedido, llamada HTTP a service-b y respuesta de inventario.

```
{service_name=~"otel-lab/service-a|otel-lab/service-b"}
```

```
{service_name=~"otel-lab/service-a|otel-lab/service-b"} |= "a4427e712044eff393ec7a28b7f10216"
```

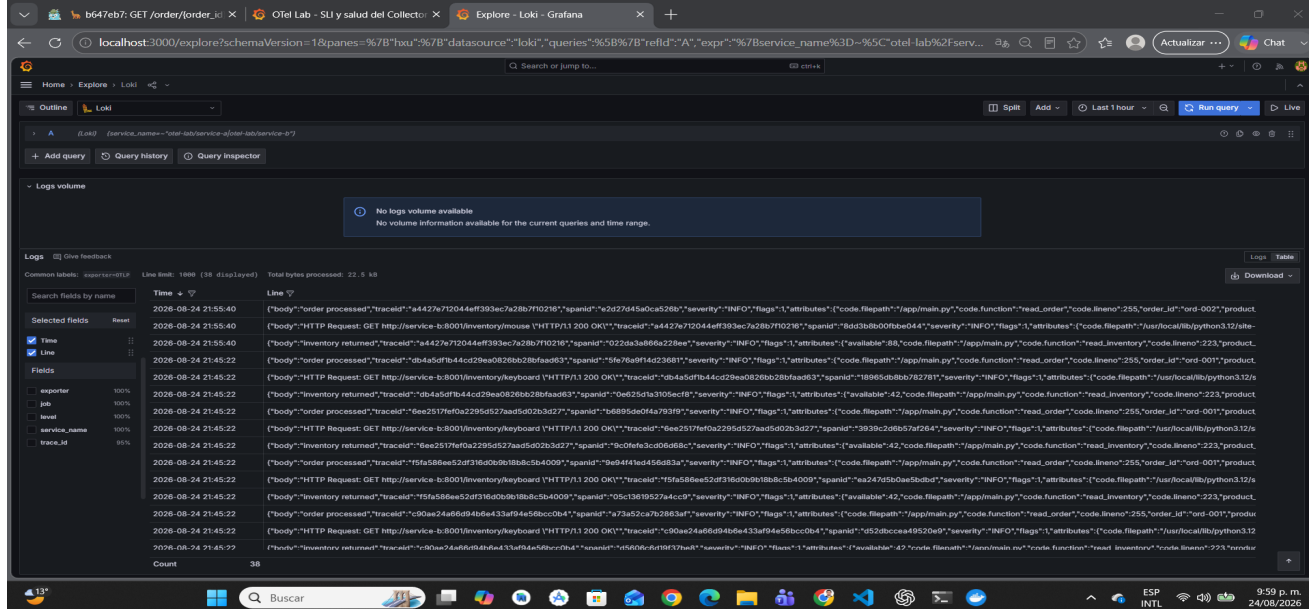


Figura 4. Logs JSON consultados en Grafana Explore con traceid y spanid.

Correlación por trace_id

La siguiente evidencia filtra los logs por el traceid a4427e712044eff393ec7a28b7f10216. El identificador coincide con la traza abierta en Jaeger, permitiendo navegar desde una línea de log hasta el timeline distribuido.

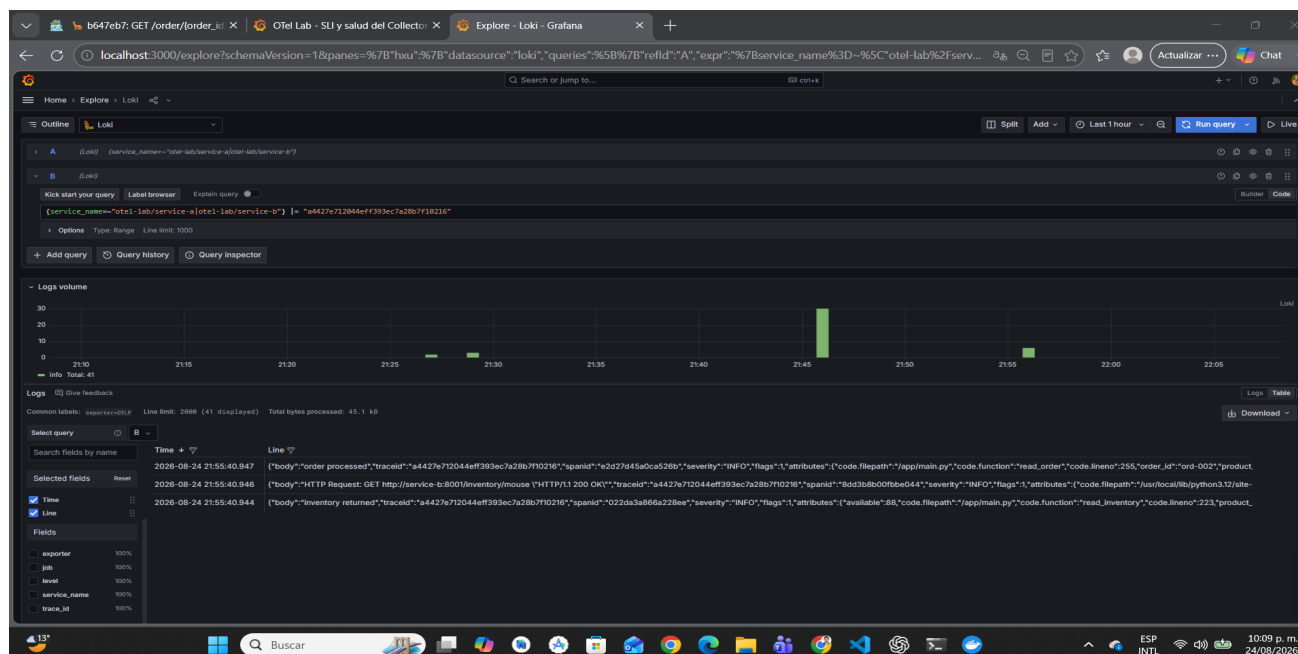


Figura 5. Consulta Loki filtrada por una traza específica.

Los archivos JSON descargados desde Loki se conservan en `evidencia/loki/`. Esto permite auditar el contenido fuera de la interfaz gráfica y reproducir la consulta durante la evaluación.

5. Benchmark y análisis de overhead

Se ejecutaron dos pruebas comparables con 50 usuarios virtuales, 30 segundos de calentamiento y cinco minutos de carga sostenida. El baseline deshabilitó el SDK OTel; la segunda ejecución habilitó trazas, métricas y logs hacia el Collector.

Métrica	Baseline	Con OTel	Delta
Latencia promedio	538.70 ms	776.29 ms	+44.11 %
p95	938.00 ms	1 293.08 ms	+37.86 %
p99	1 138.14 ms	1 585.64 ms	+39.32 %
Throughput	53.70 req/s	42.43 req/s	-20.99 %
Tasa de error	0.00 %	0.00 %	0 puntos
CPU promedio total	0.810 %	1.969 %	+143.09 %
Memoria promedio total	102.79 MiB	103.12 MiB	+0.33 MiB

Interpretación

Con OTel, el p99 aumentó 447.50 ms y el throughput disminuyó 20.99 %, pero no se observaron errores funcionales. El aumento relativo de CPU debe interpretarse con cautela porque el baseline consume muy poco y presentó un pico aislado de 2.75 % en service-b. La memoria aumentó solo 0.33 MiB en promedio para los dos servicios.

Las mediciones fueron tomadas en contenedores Docker sobre Windows y sirven como evidencia del laboratorio, no como una proyección exacta de producción en GKE o ECS Fargate.

6. Alcance, rúbrica y referencias

Esta entrega fue ejecutada y validada localmente con Docker Desktop. No se aplicaron recursos en GCP ni AWS porque no había cuentas disponibles y la indicación recibida en clase fue realizar las mediciones localmente. El resultado cloud queda documentado como IaC de referencia en `infra/gcp` e `infra/aws`, pero no se presenta como despliegue ejecutado.

La evidencia principal se organiza de acuerdo con los criterios de evaluación:

Criterio	Evidencia entregada
Instrumentación OTel SDK	SDK de Python, auto-instrumentación HTTP/DB, spans personalizados, métricas, logs JSON y trazas.
OTel Collector	Receiver OTLP gRPC/HTTP, memory_limiter, resource, batch y exporters Jaeger, Prometheus, Loki y debug.
Correlación cross-signal	Jaeger, logs JSON en Loki y capturas con el mismo trace_id entre service-a y service-b.
Benchmark de overhead	Baseline y OTel con 50 VUs, cinco minutos, resultados k6, CPU, memoria y tabla comparativa.
IaC y calidad	Terraform de referencia, Docker Compose reproducible, documentación, capturas, JSON, TXT y reporte PDF.

Inventario de evidencias

Capturas: Jaeger, propagación del trace ID, dashboard de seis paneles, logs JSON, correlación Loki y consultas Prometheus. Datos: `evidence/prometheus/*.json` y `evidence/loki/*.json`. Transcripciones: `evidence/benchmark/*.txt`. Documentación: `evidence/collector-healthy.md`, evidencia Prometheus y análisis de overhead. El README contiene los enlaces directos a cada artefacto.

Referencias técnicas

- OpenTelemetry Python SDK - <https://opentelemetry-python.readthedocs.io/>
- Jaeger Architecture Documentation - <https://www.jaegertracing.io/docs/architecture/>
- Grafana Trace Integration - <https://grafana.com/docs/grafana/latest/explore/trace-integration/>
- Grafana k6 Documentation - <https://k6.io/docs/>
- W3C Trace Context Specification - <https://www.w3.org/TR/trace-context/>

7. Conclusiones, limitaciones y reproducción

El objetivo se cumplió: la aplicación emite las tres señales, el Collector centraliza su procesamiento, Jaeger permite analizar la traza completa, Prometheus alimenta el dashboard y Loki conserva logs JSON correlacionables. El `trace_id` se mantiene entre `service-a`, `service-b`, PostgreSQL y los logs asociados.

La evidencia local evita costos de GCP y AWS. El diseño cloud queda representado en `infra/gcp` y `infra/aws` para una eventual migración a GKE y ECS Fargate, pero no se aplicó infraestructura durante esta entrega.

Limitaciones

Jaeger utiliza memoria y no representa alta disponibilidad ni retención productiva. El benchmark se ejecutó sobre Docker Desktop y la medición de CPU depende del host Windows. Para producción se requerirían TLS, autenticación OTLP, gestión de secretos, retención, alertas, escalamiento y pruebas con una infraestructura representativa.

Comandos principales de reproducción

```
docker compose up -d --build
Invoke-RestMethod http://localhost:8000/order/ord-001
docker compose ps
docker compose logs --since=5m --no-color otel-collector
py benchmark/analyze_overhead.py
```

Artefactos incluidos

- `docker-compose.yaml` y `docker-compose.baseline.yaml`
- `service-a/`, `service-b/` y `scripts/init-db.sql`
- `otel-collector/collector-config.yaml`
- `prometheus/`, `grafana/` y `loki/`
- `evidence/` con JSON, transcripciones y explicaciones Markdown
- `screenshots/` con trazas, dashboard, logs y Prometheus
- `benchmark/raw/` con los resultados JSON
- `infra/gcp/` e `infra/aws/` con parametrización Terraform

Conclusión final

OpenTelemetry aporta visibilidad integral y correlación entre señales a cambio de un overhead medible. En este laboratorio el costo principal se refleja en latencia y throughput, mientras que la memoria adicional es reducida y la tasa de errores se mantiene en cero. La capacidad de recorrer una incidencia desde Grafana o un log hasta una traza completa mejora la resiliencia y la capacidad de diagnóstico del sistema.